

UNIVERSIDADE DO MINHO

Licenciatura em Engenharia Informática



Computação Gráfica

2025/2026

Fase 1

Primitivas Gráficas

GRUPO 22

106804, Alice Isabel Faria Soares

106853, Ana Beatriz Freitas

107365, Beatriz Martins Miranda

107367, João Paulo Batista Azevedo

Fevereiro 2026

Índice

1	Introdução	2
2	Arquitetura Geral	2
2.1	Estrutura de ficheiros	2
2.2	Fluxo de dados	2
3	Generator	3
3.1	Formato dos Ficheiros .3d	3
3.2	Primitivas Gráficas	3
3.2.1	Plano	4
3.2.2	Caixa	5
3.2.3	Esfera	7
3.2.4	Cone	9
3.2.5	Cilindro	11
3.2.6	Torus	13
4	Engine	15
4.1	Arquitetura e separação de responsabilidades	15
4.2	Estruturas de dados	15
4.3	Parser XML	15
4.4	Formato do ficheiro XML de cena	16
4.5	Modelo de câmara	17
4.5.1	Dois modos de interação	17
4.6	Rendering	18
4.7	Tabela de controlos	19
5	Resultados dos Testes	19
6	Conclusão	21

1 Introdução

O presente relatório apresenta as decisões e as abordagens adotadas no desenvolvimento da primeira fase do trabalho prático da unidade curricular de Computação Gráfica.

Nesta fase, foram desenvolvidos dois programas independentes: o *generator*, responsável pela criação de ficheiros de geometria a partir de primitivas gráficas, e o *engine*, encarregado de ler uma cena descrita em formato XML e proceder à sua renderização em modo *wireframe*, com suporte a navegação interativa.

O projeto foi desenvolvido em C++17, com recurso às bibliotecas *OpenGL*, *freeGLUT* e *expat*, e compilado com *CMake*.

2 Arquitetura Geral

O projeto está dividido em dois programas principais independentes, cada um com responsabilidades distintas e estrutura de ficheiros separada.

O *generator* gera figuras primitivas geométricas com base em parâmetros fornecidos e guarda os dados resultantes em ficheiros *.3d*. Esses ficheiros são posteriormente referenciados em ficheiros *.xml*, que são lidos pelo *engine*, responsável por interpretar a *scene* e efetuar a sua renderização.

2.1 Estrutura de ficheiros

O projeto está dividido em várias diretorias bem definidas:

- **engine/** – código fonte do motor de *render*, lê as *scenes* XML e desenha os modelos;
- **generator/** – programa que cria as primitivas e escreve ficheiros *.3d*;
- **models/** – modelos 3D pré-calculados das primitivas (ex. ‘box.3d’);
- **scenes/** – definições XML das *scenes*;
- **test files/** – pastas com os cenários usados nas várias fases *phase_n/* para teste.

2.2 Fluxo de dados

O fluxo de dados entre as duas aplicações é unidirecional. O *generator* produz os ficheiros *.3d* relativos às primitivas e, durante a inicialização, o *engine* carrega uma *scene* XML que referencia esses ficheiros, renderizando-a com *OpenGL/GLUT*.

3 Generator

O *generator* é responsável por gerar modelos .3d, obtidos através de cálculos, que serão posteriormente renderizados pelo *engine*.

Este segue o princípio de responsabilidade única, onde cada primitiva geométrica tem o seu próprio par de ficheiros (.h/.cpp), e a lógica de escrita para disco está, ainda, isolada no módulo *filewriter*. Desta forma, adicionar novas primitivas nas próximas fases não requererá modificação dos ficheiros existentes.

Quando utilizado, recebe como argumentos o nome da figura a gerar, os parâmetros específicos dessa figura e o nome do ficheiro .3d onde os vértices serão armazenados. O resultado é um ficheiro que contém a listagem sequencial dos vértices que definem os triângulos do modelo.

Cada primitiva suporta um conjunto próprio de parâmetros, conforme descrito na Tabela 1.

Modelo	Parâmetros
plane	length (float), divisions (int)
box	size (float), divisions (int)
cone	radius (float), height (float), slices (int), stacks (int)
sphere	radius (float), slices (int), stacks (int)
cylinder	radius (float), height (float), slices (int), stacks (int), capped (bool)
torus	outerRadius (float), innerRadius (float), sides (int), rings (int)

Tabela 1: Parâmetros das primitivas geométricas suportadas pelo *generator*

3.1 Formato dos Ficheiros .3d

Nesta fase, definimos o formato .3d como um ficheiro de texto simples, onde a primeira linha do ficheiro contém um número inteiro correspondente ao número total de vértices e cada uma das linhas seguintes representa exatamente um vértice contendo oito valores numéricos separados por espaços:

x y z nx ny nz u v

onde:

- (x, y, z) representam a posição do vértice no espaço;
- (n_x, n_y, n_z) correspondem ao vetor normal associado;
- (u, v) são as coordenadas de textura.

Cada grupo de três vértices consecutivos define um triângulo. Esta abordagem simplifica a leitura no *engine*, pois basta percorrer sequencialmente o ficheiro e enviar os vértices para o *pipeline* gráfico.

A inclusão dos oito campos desde a Fase 1 foi uma decisão deliberada. Embora numa fase inicial os campos da textura assumam valores nulos, esta abordagem evita grandes alterações ao formato e ao *parser* em fases posteriores.

3.2 Primitivas Gráficas

Além das primitivas obrigatórias, plano, caixa, esfera e cone, decidimos implementar também o **cilindro** e o **torus**. Todas as primitivas usam ordem de vértices CCW vista do exterior, em conformidade com a convenção OpenGL para faces frontais. As normais estão calculadas analiticamente e armazenadas nos ficheiros .3d desde já, ainda que apenas sejam utilizadas pelo engine numa fase posterior.

3.2.1 Plano

O plano é gerado no plano XZ , centrado na origem, com lado de comprimento $length$ e subdividido uniformemente em $divisions$ partes ao longo de cada eixo. Esta subdivisão origina uma grelha composta por células quadradas, que por sua vez são compostas por dois triângulos. O número de células da grelha é dado por:

$$N_{\text{células}} = divisions \times divisions = divisions^2$$

Para determinar os vértices dos triângulos, calculamos o passo da grelha, isto é, o tamanho de cada subdivisão, dado por:

$$step = \frac{length}{divisions}$$

Como o plano está centrado na origem, calculamos a metade do comprimento do lado, pois as coordenadas variam ao longo dos eixos X e Z no intervalo:

$$[-half, half], \quad half = \frac{length}{2}$$

O plano é construído iterando sobre as divisões ao longo dos eixos X e Y . Para cada célula identificada pelos índices (i, j) , os quatro vértices são:

$$\begin{aligned} A &= (-half + i \times step, 0, -half + j \times step + step) \\ B &= (-half + i \times step + step, 0, -half + j \times step + step) \\ C &= (-half + i \times step, 0, -half + j \times step) \\ D &= (-half + i \times step + step, 0, -half + j \times step) \end{aligned}$$

Para garantir a visualização do plano em ambas as faces, é necessário definir duas normais em sentidos opostos: uma com direção $+Y$ (face superior) e outra com direção $-Y$ (face inferior).

Definindo a ordem dos vértices em sentido anti-horário (CCW), vistos de cima, os dois triângulos emitidos para cada célula são (A, B, C) e (C, B, D) , com norma $(0,1,0)$. Já para a face inferior, mantendo a orientação CCW, foram emitidos os triângulos (C, B, A) e (D, B, C) , com norma $(0,-1,0)$.

Resultando num total de:

$$\begin{aligned} N_{\text{triângulos por face}} &= 2 \times divisions^2 \\ N_{\text{triângulos totais}} &= 4 \times divisions^2 \end{aligned}$$

Como cada triângulo é definido por três vértices independentes, o número total de vértices gerados é:

$$N_{\text{vértices}} = 3 \times 4 \times divisions^2 = 12 \times divisions^2$$

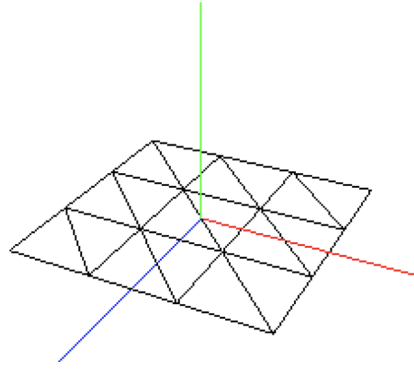


Figura 1: Plano

3.2.2 Caixa

A caixa é gerada centrada na origem do referencial, sendo definida por dois parâmetros: o comprimento total da aresta, *size*, e o número de subdivisões em cada face, *divisions*.

Esta é criada face a face, onde cada uma das seis faces (Topo, Fundo, Frente, Trás, Direita e Esquerda) é tratada como um plano independente com normal constante orientada para o exterior, seguindo uma lógica idêntica à utilizada na geração do plano.

As seis normais das faces são:

$$\hat{n} \in \{(0, 1, 0), (0, -1, 0), (0, 0, 1), (0, 0, -1), (1, 0, 0), (-1, 0, 0)\}$$

Calculamos, então, a distância do centro da caixa a cada face, dada pelo semi-comprimento da aresta:

$$h = \frac{size}{2}$$

E o tamanho de cada subdivisão da grelha:

$$step = \frac{size}{divisions}$$

Para cada célula identificada pelos índices (i, j) , calculam-se os limites da subdivisão:

$$a = -h + i \times step$$

$$b = a + step$$

$$c = -h + j \times step$$

$$d = c + step$$

Os valores a e b representam os limites da célula na primeira direção da face, isto é, na direção associada ao índice i , enquanto c e d representam os limites na segunda direção, isto é, na direção associada ao índice j .

Assim, para cada célula são criados quatro vértices do tipo **Vertex**, designados por a , b , c e d , que definem um *quad*. Esse *quad* é depois decomposto em dois triângulos:

$$T_1 = (a, b, c)$$

$$T_2 = (c, b, d)$$

garantindo orientação consistente (CCW) quando observado do exterior.

A diferença entre faces está assente em três aspetos fundamentais:

1. Qual o eixo que permanece fixo (igual a $+h$ ou $-h$);
2. Em que eixos são colocados os valores a, b e c, d ;
3. A orientação dos vértices, de modo a garantir que a normal aponta para o exterior da caixa.

Em cada face, dois eixos variam segundo a grelha (a, b, c, d) e o terceiro eixo permanece constante em $\pm h$.

- **Face do Topo (+Y)**

Nesta face o eixo fixo é o eixo Y , com $y = h$. A grelha desenvolve-se no plano XZ , pelo que:

$$x \in \{a, b\}, \quad z \in \{c, d\}.$$

Os quatro vértices do *quad* são:

$$(a, h, d), (b, h, d), (a, h, c), (b, h, c),$$

com normal constante $(0, 1, 0)$.

- **Face do Fundo (-Y)**

Tal como no topo, a grelha está no plano XZ , mas agora com $y = -h$. Os vértices são:

$$(b, -h, d), (a, -h, d), (b, -h, c), (a, -h, c).$$

A ordem de a e b é invertida relativamente à face superior. Esta inversão garante que a orientação dos triângulos continua a ser anti-horária quando vista do exterior, sendo que a normal é $(0, -1, 0)$.

- **Face da Frente (+Z)**

O eixo fixo é o eixo Z , com $z = +h$. A grelha desenvolve-se no plano XY , logo:

$$x \in \{a, b\}, \quad y \in \{c, d\}.$$

Os vértices são:

$$(a, c, h), (b, c, h), (a, d, h), (b, d, h),$$

com normal $(0, 0, 1)$.

- **Face de Trás (-Z)**

Idêntica à face da frente, mas com $z = -h$. Os vértices são:

$$(b, c, -h), (a, c, -h), (b, d, -h), (a, d, -h),$$

com normal $(0, 0, -1)$.

A ordem dos vértices é novamente invertida para manter a orientação exterior.

- **Face da Direita (+X)**

Nesta face o eixo fixo é X , com $x = +h$. A grelha desenvolve-se no plano YZ , pelo que:

$$y \in \{c, d\}, \quad z \in \{a, b\}.$$

Os vértices são:

$$(h, c, b), (h, c, a), (h, d, b), (h, d, a),$$

com normal $(1, 0, 0)$.

- **Face da Esquerda ($-X$)**

Semelhante à direita, mas com $x = -h$. Mantém-se o plano YZ como plano de variação:

$$(-h, c, a), (-h, c, b), (-h, d, a), (-h, d, b),$$

com normal $(-1, 0, 0)$.

Desta forma, o número total de triângulos é então:

$$N_{\text{triângulos totais}} = 6 \times N_{\text{triângulos por face do plano}} = 6 \times 2 \times \text{divisions}^2 = 12 \times \text{divisions}^2$$

Sendo cada triângulo definido por três vértices independentes:

$$N_{\text{vértices}} = 3 \times N_{\text{triângulos totais}} = 36 \times \text{divisions}^2$$

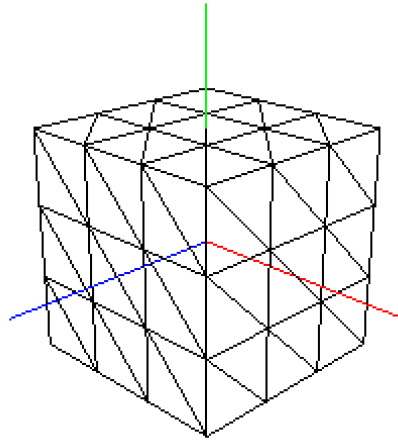


Figura 2: Caixa

3.2.3 Esfera

A esfera é gerada centrada na origem e é definida por três parâmetros: o raio, *radius*, o número de divisões longitudinais *slices* e o número de divisões latitudinais *stacks*.

A superfície da esfera é descrita por dois ângulos:

- ϕ (Phi) - ângulo de latitude, que varia no intervalo $[-\frac{\pi}{2}, \frac{\pi}{2}]$;
- θ (Theta) - ângulo de longitude, que varia no intervalo $[0, 2\pi]$.

Os incrementos angulares calculados são:

$$d\phi = \frac{\pi}{\text{stacks}}$$

$$d\theta = \frac{2\pi}{\text{slices}}$$

Para cada *stack* $i \in \{0, \dots, stacks - 1\}$ e *slice* $j \in \{0, \dots, slices - 1\}$, os ângulos da célula são:

$$\begin{aligned}\phi_0 &= -\frac{\pi}{2} + i \times d\phi, & \phi_1 &= \phi_0 + d\phi \\ \theta_0 &= j \times d\theta, & \theta_1 &= \theta_0 + d\theta\end{aligned}$$

Cada ponto da superfície é calculado em coordenadas esféricas e convertido para coordenadas cartesianas. Para um par (ϕ, θ) :

$$P(\phi, \theta) = (radius \cos(\phi) \sin(\theta), radius \sin(\phi), radius \cos(\phi) \cos(\theta))$$

Para uma esfera centrada na origem, o vetor normal em qualquer ponto da superfície coincide com o vetor posição normalizado, obtido diretamente a partir dos mesmos ângulos:

$$\hat{n}(\phi, \theta) = (\cos(\phi) \sin(\theta), \sin(\phi), \cos(\phi) \cos(\theta))$$

Desta forma, posição e a normal são obtidas simultaneamente a partir do mesmo par de ângulos, não sendo necessário qualquer cálculo adicional de produtos vetoriais.

Os quatro vértices da célula quadrangular (i, j) são:

$$\begin{aligned}v_{00} &= P(\phi_0, \theta_0) \\ v_{01} &= P(\phi_0, \theta_1) \\ v_{10} &= P(\phi_1, \theta_0) \\ v_{11} &= P(\phi_1, \theta_1)\end{aligned}$$

Cada célula é decomposta nos triângulos (v_{00}, v_{01}, v_{10}) e (v_{01}, v_{11}, v_{10}) , em ordem CCW vista do exterior. Esta construção é repetida para todas as combinações de *slices* e *stacks*, garantindo o fecho completo da superfície.

Mais uma vez, cada célula origina dois triângulos, logo:

$$N_{\text{triângulos}} = 2 \times slices \times stacks$$

E, consequentemente:

$$N_{\text{vértices}} = 3 \times N_{\text{triângulos}} = 6 \times slices \times stacks$$

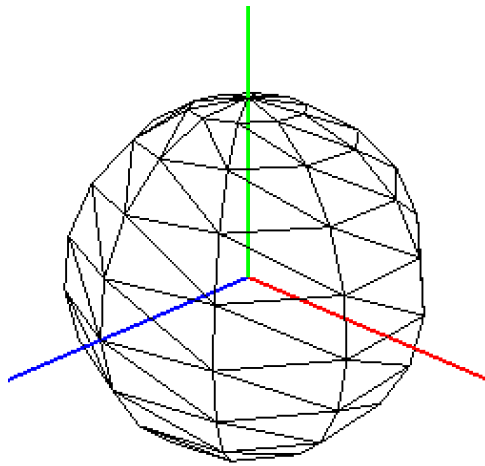


Figura 3: Esfera

3.2.4 Cone

O cone é gerado com o eixo coincidente com o eixo Y , sendo a base situada em $y = 0$ e o ápice em $y = height$. É definido por quatro parâmetros: o raio da base *radius*, a altura *height*, o número de divisões angulares *slices* e o número de divisões verticais *stacks*.

A divisão angular em torno do eixo Y é dada por:

$$d\theta = \frac{2\pi}{slices}$$

A divisão vertical é feita através de um parâmetro normalizado $t \in [0, 1]$, que representa a posição relativa ao longo da altura, cujo incremento por *stack* é:

$$dStack = \frac{1}{stacks}$$

A geração da geometria é feita em duas fases distintas. Primeiro constrói-se a **superfície lateral**, onde cada célula (i, j) define um quadrilátero paramétrico que é decomposto em dois triângulos com orientação CCW vista do exterior.

De seguida, gera-se a **base circular** como um leque de *slices* triângulos, todos com vértice comum no centro $(0, 0, 0)$ e normal constante $(0, -1, 0)$, garantindo fecho geométrico do sólido.

Superfície lateral:

A normal lateral é calculada como uma combinação adequada das radial e vertical, dependente do ângulo de inclinação do cone, garantindo vetores unitários orientados para o exterior. Seja a aresta oblíqua:

$$slant = \sqrt{radius^2 + height^2}$$

A direção radial da normal depende apenas de θ , enquanto a inclinação depende da razão entre raio e altura.

Define-se:

$$\sin \alpha = \frac{radius}{slant} \qquad \cos \alpha = \frac{height}{slant}$$

A normal unitária em cada ponto lateral é então:

$$\hat{n}_{lat}(\theta) = (\cos \alpha \sin \theta, \sin \alpha, \cos \alpha \cos \theta)$$

Para cada nível $i \in \{0, \dots, stacks - 1\}$ e setor angular $j \in \{0, \dots, slices - 1\}$, os parâmetros da célula (i, j) são:

$$\begin{aligned} t_0 &= i \times dStack, & t_1 &= t_0 + dStack, \\ y_0 &= height \times t_0, & y_1 &= height \times t_1, \\ r_0 &= radius \times (1 - t_0), & r_1 &= radius \times (1 - t_1), \\ \theta_0 &= j \times d\theta, & \theta_1 &= \theta_0 + d\theta. \end{aligned}$$

Onde:

- t_0 e t_1 são parâmetros normalizados ao longo da altura do cone. Temos $t = 0$ na base e $t = 1$ no vértice, correspondendo à posição relativa da altura.
- y_0 e y_1 correspondem às posições reais no eixo Y , sendo obtidas multiplicando t por *height*.
- r_0 e r_1 representam os raios das secções horizontais nos níveis t_0 e t_1 . No cone, o raio decresce linearmente desde *radius* (na base, $t = 0$) até 0 (no ápice, $t = 1$), garantindo a forma cônica.
- θ_0 e θ_1 são os ângulos que delimitam o setor circular correspondente a cada *slice*, variando uniformemente em incrementos de $d\theta$.

A partir destes parâmetros, a posição de um vértice é obtida por:

$$P(r, y, \theta) = (r \sin \theta, y, r \cos \theta)$$

Os quatro vértices da célula quadrangular (i, j) na superfície lateral são:

$$v00 = P(r_0, y_0, \theta_0)$$

$$v01 = P(r_0, y_0, \theta_1)$$

$$v10 = P(r_1, y_1, \theta_0)$$

$$v11 = P(r_1, y_1, \theta_1)$$

Cada célula é decomposta nos triângulos $(v00, v01, v10)$ e $(v01, v11, v10)$, em ordem CCW vista do exterior.

Assim, o número de triângulos da superfície lateral é:

$$N_{lat} = 2 \times slices \times stacks$$

Base circular:

A normal da base é constante e igual a $(0, -1, 0)$.

A base circular é gerada como um leque de *slices* triângulos, cada um com o vértice central em $(0, 0, 0)$ e os dois vértices periféricos em:

$$P_j = (radius \sin(\theta_0), 0, radius \cos(\theta_0))$$

$$P_{j+1} = (radius \sin(\theta_1), 0, radius \cos(\theta_1))$$

Onde:

$$\theta_0 = j * d\theta;$$

$$\theta_1 = \theta_0 + d\theta;$$

O número de triângulos da base é:

$$N_{base} = slices$$

Assim, o número total de triângulos é:

$$N_{\text{triângulos}} = 2 \times \text{slices} \times \text{stacks} + \text{slices} = \text{slices} (2 \text{ stacks} + 1)$$

Consequentemente, o número total de vértices é:

$$N_{\text{vértices}} = 3 \cdot \text{slices} (2 \text{ stacks} + 1)$$

Logo:

$$N_{\text{triângulos}} = 2 \times \text{slices} \times \text{stacks} + \text{slices} = \text{slices}(2 \text{ stacks} + 1)$$

$$N_{\text{vértices}} = 3 \times \text{slices}(2 \text{ stacks} + 1)$$

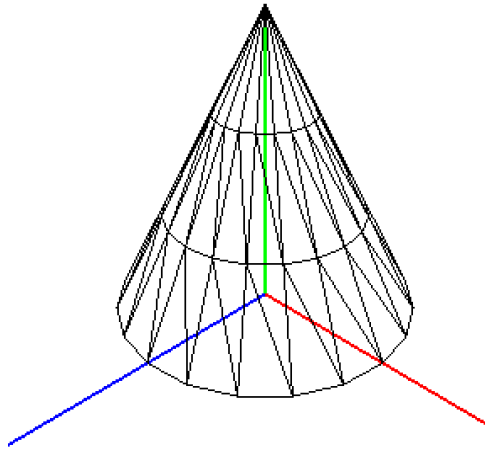


Figura 4: Cone

3.2.5 Cilindro

O cilindro é gerado centrado na origem e com a base em XY , isto é, com o eixo coincidente com o eixo Y , com a base inferior em $y = 0$ e a base superior em $y = h$. É definido por quatro parâmetros: o raio *radius*, a altura *height*, o número de divisões angulares *slices* e o número de divisões verticais *stacks* e um parâmetro booleano *capped*, que indica se o sólido inclui tampas.

A divisão angular em torno do eixo Y é dada por:

$$d\theta = \frac{2\pi}{\text{slices}}$$

O incremento de altura por *stack* é:

$$d\text{Stack} = \frac{h}{\text{stacks}}$$

Para cada *stack* $i \in \{0, \dots, \text{stacks} - 1\}$ e setor angular $j \in \{0, \dots, \text{slices} - 1\}$, os parâmetros da célula são:

$$\begin{aligned} y_0 &= i \times d\text{Stack}, & y_1 &= y_0 + d\text{Stack} \\ \theta_0 &= j \times d\theta, & \theta_1 &= \theta_0 + d\theta \end{aligned}$$

Semelhante ao cone, a posição de qualquer vértice lateral é dada por:

$$P(radius, y, \theta) = (radius \sin(\theta), y, radius \cos(\theta))$$

Os quatro vértices da célula quadrangular (i, j) na superfície lateral são:

$$v00 = P(radius, y_0, \theta_0)$$

$$v01 = P(radius, y_0, \theta_1)$$

$$v10 = P(radius, y_1, \theta_0)$$

$$v11 = P(radius, y_1, \theta_1)$$

Cada célula é decomposta nos triângulos $(v00, v01, v10)$ e $(v01, v11, v10)$, em ordem CCW vista do exterior.

A normal é puramente radial:

$$\hat{n}_{lat}(\theta) = (\sin(\theta), 0, \cos(\theta))$$

Se *capped* for verdadeiro, as tampas são geradas como leques de *slices* triângulos cada, a partir do centro, com os vértices periféricos em:

$$a = (radius \sin(\theta_0), y_{tampa}, radius \cos(\theta_0))$$

$$b = (radius \sin(\theta_1), y_{tampa}, radius \cos(\theta_1))$$

onde $y_{tampa} = 0$ para a tampa inferior e $y_{tampa} = h$ para a tampa superior. As normais são constantes: $(0, -1, 0)$ e $(0, 1, 0)$, respetivamente. A ordem dos vértices em cada triângulo é invertida entre tampas para garantir orientação CCW vista do exterior.

O número total de triângulos e vértices é:

$$N_{triângulos} = 2 \times slices \times stacks + 2 \times slices = 2 slices (stacks + 1)$$

$$N_{vértices} = 6 slices (stacks + 1)$$

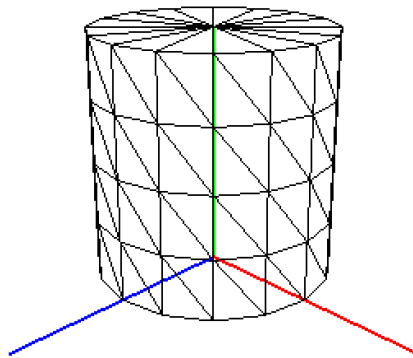


Figura 5: Cilindro

3.2.6 Torus

O *torus* é gerado centrado na origem, e é definido pelo número de divisões ao longo do anel principal, *rings*, pelo número de divisões ao longo do tubo, *sides* e por dois raios:

- *outerRadius*: distância da origem até ao centro da secção circular do tubo, ou seja, determina onde, radialmente, o tubo se encontra em relação à origem;
- *innerRadius*: raio da secção circular do tubo. Determina a espessura do tubo em torno do seu eixo local.

Para que exista um buraco central

$$0 < innerRadius < outerRadius$$

O torus é parametrizado por dois ângulos:

- $\phi(phi) \in [0, 2\pi]$ - percorre a secção ao longo do tubo em torno do eixo *Y*
- $\theta(theta) \in [0, 2\pi]$ - percorre a secção circular do tubo, descrevendo a circunferência da secção.

$$0 < innerRadius < outerRadius$$

Os incrementos angulares calculados são:

$$d\phi = \frac{2\pi}{rings}$$

$$d\theta = \frac{2\pi}{sides}$$

A posição de um ponto na superfície é dada por :

$$P(\phi, \theta) = \begin{pmatrix} (outerRadius + innerRadius \times \cos(\theta)) \times \cos(\phi) \\ innerRadius \times \sin \theta \\ (outerRadius + innerRadius \times \cos(\theta)) \times \sin(\phi) \end{pmatrix}$$

Para cada célula (i, j) da malha, definida pelos ângulos

$$\begin{aligned} \phi_0 &= i \times d\phi, & \phi_1 &= \phi_0 + d\phi \\ \theta_0 &= j \times d\theta, & \theta_1 &= \theta_0 + d\theta \end{aligned}$$

os quatro vértices da célula são:

$$\begin{aligned} v_{00} &= P(\phi_0, \theta_0) \\ v_{10} &= P(\phi_1, \theta_0) \\ v_{01} &= P(\phi_0, \theta_1) \\ v_{11} &= P(\phi_1, \theta_1) \end{aligned}$$

Cada célula é decomposta nos triângulos (v_{00}, v_{01}, v_{11}) e (v_{00}, v_{11}, v_{10}) , em ordem CCW vista do exterior.

A normal aponta do centro do tubo para o ponto da superfície:

$$\hat{n}(\phi, \theta) = (\cos(\theta) \cos(\phi), \sin(\theta), \cos(\theta) \sin(\phi))$$

O número total de triângulos e vértices é:

$$N_{\text{triângulos}} = 2 \times \text{rings} \times \text{sides}$$

$$N_{\text{vértices}} = 6 \times \text{rings} \times \text{sides}$$

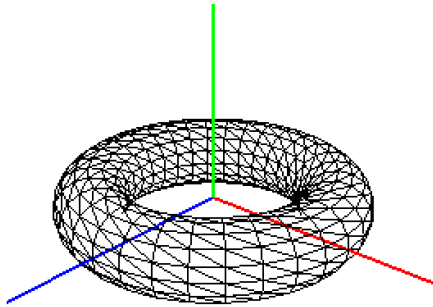


Figura 6: *Torus*

4 Engine

4.1 Arquitetura e separação de responsabilidades

O *engine* está dividido em quatro módulos com responsabilidades bem definidas, de modo a minimizar as alterações necessárias em cada fase do projeto. A forma como estes módulos comunicam é observável na seguinte figura:

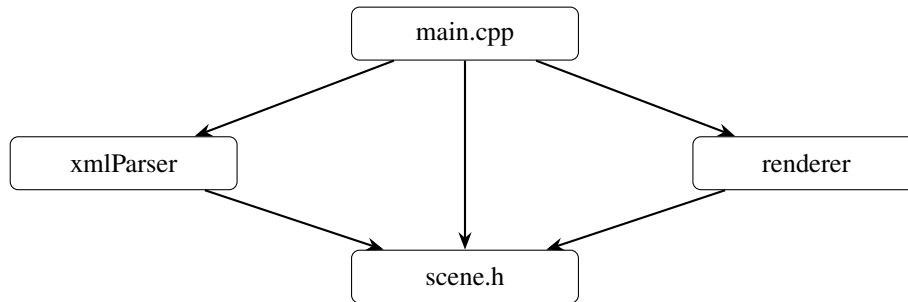


Figura 7: Dependências entre módulos do engine e fases em que cada um é alterado.

4.2 Estruturas de dados

A *scene* é representada por uma hierarquia de estruturas que reflete o grafo de *scene* do enunciado.

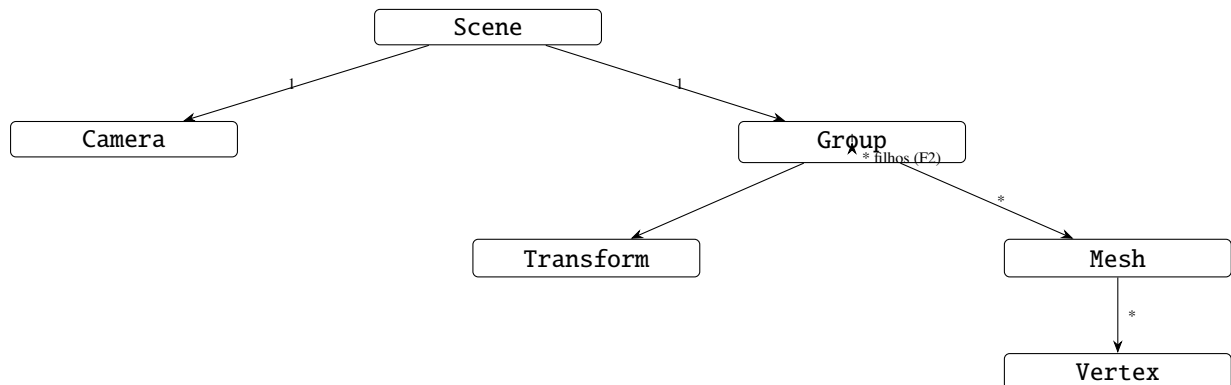


Figura 8: Diagrama de estruturas de dados da cena. A recursividade em Group é usada a partir da Fase 2.

Cada Vertex contém os oito campos do formato .3d:

$$v = (x, y, z, n_x, n_y, n_z, u, tv)$$

4.3 Parser XML

A biblioteca escolhida para o parsing XML foi o **expat**, um parser SAX (*Simple API for XML*). Ao contrário dos parsers DOM (como o `tinyxml2`), o expat processa o documento de forma incremental, invocando *callbacks* à medida que encontra cada elemento. Para o caso de uso deste projeto (percorrer o documento uma única vez do início ao fim) esta abordagem é mais eficiente em memória do que construir uma árvore completa. Outro motivo da escolha foi a disponibilidade como pacote de sistema, sem necessidade de incluir código de terceiros no repositório.

O diagrama seguinte descreve o fluxo de parsing:

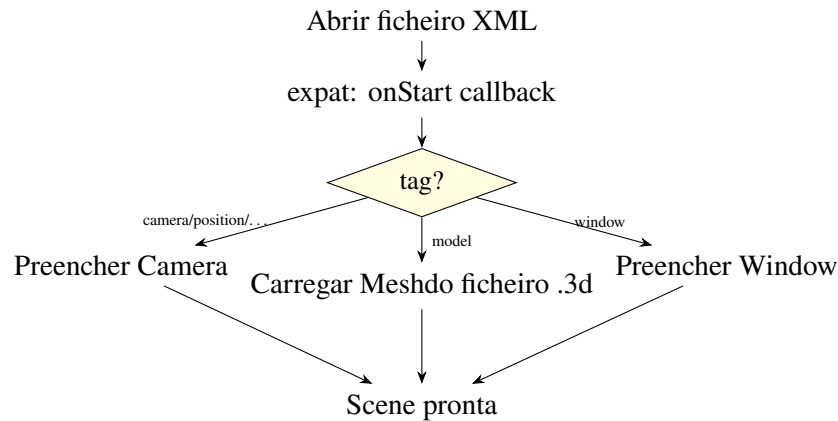


Figura 9: Fluxo de parsing XML com expat.

4.4 Formato do ficheiro XML de cena

O ficheiro XML de entrada segue a estrutura apresentada abaixo. A tag raiz é `<world>`, que contém obrigatoriamente as tags `<window>` e `<camera>`, seguidas de um ou mais `<group>` com os modelos a renderizar.

Listing 1: Exemplo de ficheiro XML de cena com uma caixa.

```

1 <world>
2   <window width="512" height="512" />
3   <camera>
4     <position x="5" y="3" z="5" />
5     <lookAt x="0" y="0" z="0" />
6     <up x="0" y="1" z="0" />
7     <projection fov="60" near="1" far="1000" />
8   </camera>
9   <group>
10    <models>
11      <model file="../models/box.3d" />
12    </models>
13  </group>
14 </world>

```

- `<window>` - define a resolução da janela em píxeis.
- `<camera>` - define:
 - `<position>` - a posição inicial da câmara.
 - `<lookAt>` - o ponto para onde esta aponta
 - `<up>` - vetor up
 - `<projection>` - os parâmetros (fov, near e far).
- `<group>` - contém uma lista de modelos:
 - `<model>` - referencia o caminho relativo para o respectivo ficheiro .3d.

4.5 Modelo de câmara

A câmara é definida por coordenadas esféricas (α, β, r) , onde α é o ângulo horizontal, β o ângulo vertical e r a distância ao ponto de interesse (*look at point* $\vec{L}$). A posição cartesiana é calculada como:

$$\vec{P} = \vec{L} + r \vec{D} \quad (1)$$

onde $\vec{L}$ é o *look at point* e $\vec{D}$ é o vetor de direção normalizado:

$$\vec{D} = \begin{pmatrix} \cos \beta \sin \alpha \\ \sin \beta \\ \cos \beta \cos \alpha \end{pmatrix} \quad (2)$$

Ou seja,

$$\vec{P} = \begin{pmatrix} L_x + r \cos \beta \sin \alpha \\ L_y + r \sin \beta \\ L_z + r \cos \beta \cos \alpha \end{pmatrix} \quad (3)$$

Para evitar que a câmara fique invertida (*upside down*), o ângulo vertical é limitado:

$$|\beta| \leq 1,5 \text{ rad} \approx 85,9 \quad (4)$$

O vetor *up* é fixo em $(0, 1, 0)$, garantido como seguro pela restrição (4), que impede que $\vec{D}$ fique paralelo ao eixo Y .

4.5.1 Dois modos de interação

A câmara suporta dois modos de movimento distintos, Figura 10:

Modo orbital: usando as teclas W/S/A/D, Q/E e rato, $\vec{L}$ permanece fixo e a câmara orbita à sua volta alterando α e β , ou aproxima/afasta alterando r .

Modo forward/backward (freecam): usando as setas e teclas Z/X, a câmara translada k unidades e tanto $\vec{P}$ como $\vec{L}$ são deslocados, preservando a orientação:

$$\vec{P}' = \vec{P} \pm k \vec{v}, \quad \vec{L}' = \vec{L} \pm k \vec{v}$$

onde + corresponde a frente/direita/cima e – ao movimento oposto (trás/esquerda/baixo), usando o mesmo vector $\vec{v}$ com sinal invertido, sendo os vetores de movimento definidos em função de α :

$$\vec{v}_{\text{frente}} = \begin{pmatrix} \sin \alpha \\ 0 \\ \cos \alpha \end{pmatrix}, \quad \vec{v}_{\text{direita}} = \begin{pmatrix} \cos \alpha \\ 0 \\ -\sin \alpha \end{pmatrix}, \quad \vec{v}_{\text{cima}} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$$

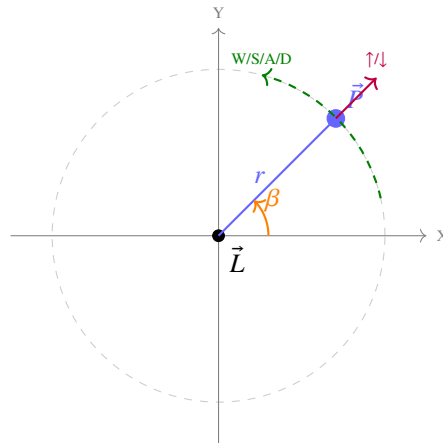


Figura 10: Modelo de câmara esférica. A órbita (verde) altera α , β ou r mantendo $\vec{L}$ fixo (3). A translação (roxo) desloca $\vec{P}$ e $\vec{L}$ em conjunto ao longo de $\vec{v}$ (4.5.1).

4.6 Rendering

O engine suporta três modos de visualização, cicláveis com a tecla M:

WIREFRAME - Apenas os triângulos são desenhados a preto.

SOLID - Os triângulos são preenchidos a cinzento claro.

SOLID_WIRE - Combinação de dois tipos de rendering sobre a mesma geometria:

1. **Fill** — preenchimento a azul-aço com offset activado, para afastar ligeiramente as faces em profundidade e evitar *z-fighting* com o wireframe sobreposto.
2. **Wireframe** — arestas a verde, desenhadas por cima sem offset.

O face culling (`GL_CULL_FACE`, `GL_BACK`) é activado na inicialização e mantém-se activo em todos os modos. O winding order CCW está definido correctamente em todas as primitivas geradas, garantindo que nenhuma face visível é removida indevidamente.

Adicionalmente, a tecla B activa e desactiva a visualização dos eixos XYZ no referencial do mundo: X a vermelho, Y a verde e Z a azul. Os eixos são desenhados depois da geometria da cena, independentemente do modo de rendering activo.

A cena é redesenhada apenas quando há interação do utilizador, via `glutPostRedisplay()`. Não é usada uma *idle function*, evitando redeseños desnecessários em cenas estáticas.

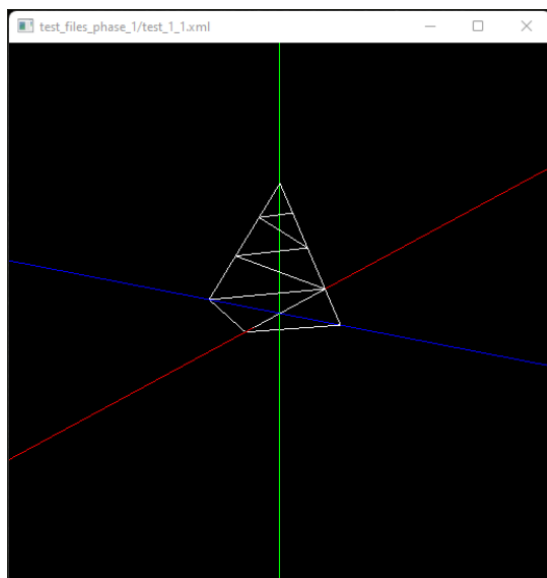
4.7 Tabela de controlos

Controlo	Ação
W / S	Órbita vertical ($\beta \pm 0,05$ rad)
A / D	Órbita horizontal ($\alpha \pm 0,05$ rad)
Q / E	Zoom in / out ($r \times = 0,95 / 1,05$)
Scroll do rato	Zoom in / out ($r \times = 0,95 / 1,05$)
Rato (botão esquerdo + arrastar)	Órbita livre (altera α e β)
Seta $\uparrow / \downarrow$	Avanço / recuo ao longo de $\vec{v}_{\text{frente}}$
Seta $\leftarrow / \rightarrow$	Deslocamento lateral ao longo de $\vec{v}_{\text{direita}}$
Z / X	Subir / descer ao longo de $\vec{v}_{\text{cima}}$
M	Cicla modo de renderização (Wireframe $\rightarrow$ Solid $\rightarrow$ Solid+Wireframe)
B	Activa / desactiva visualização dos eixos XYZ
R	Repõe a câmara na posição definida no XML
ESC	Sair da aplicação

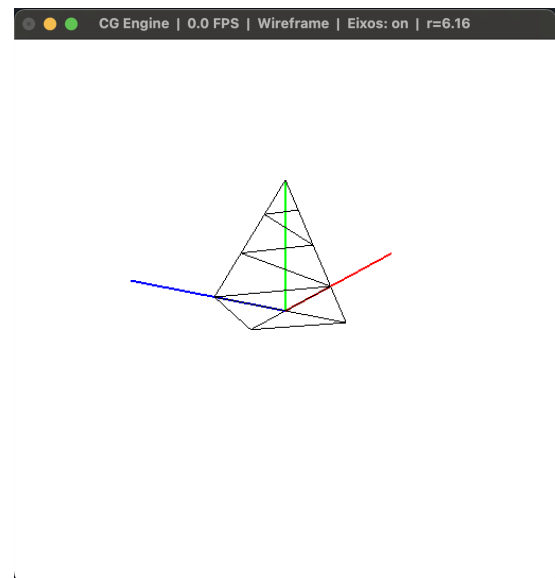
Tabela 2: Controlos de interacção do engine.

5 Resultados dos Testes

Nesta secção são apresentados os resultados obtidos pelo *engine* para os cinco testes da Fase 1, comparados com os resultados esperados fornecidos pelo enunciado. Em todos os casos, a renderização foi efetuada em modo *wireframe*, com os eixos XYZ visíveis (vermelho X, verde Y, azul Z).

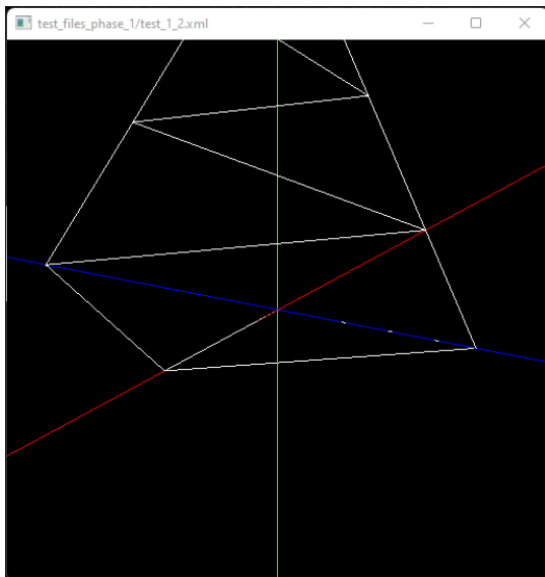


(a) Resultado Esperado

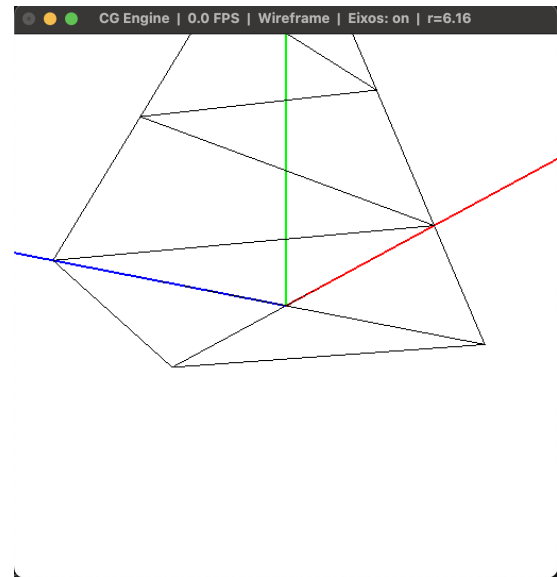


(b) Resultado Obtido

Figura 11: Teste 1

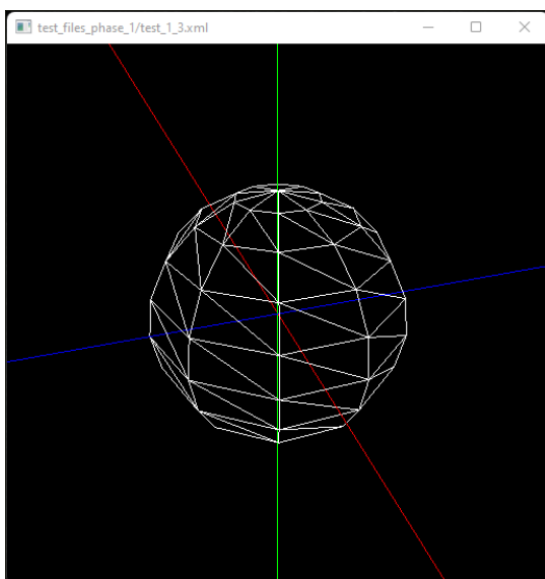


(a) Resultado Esperado

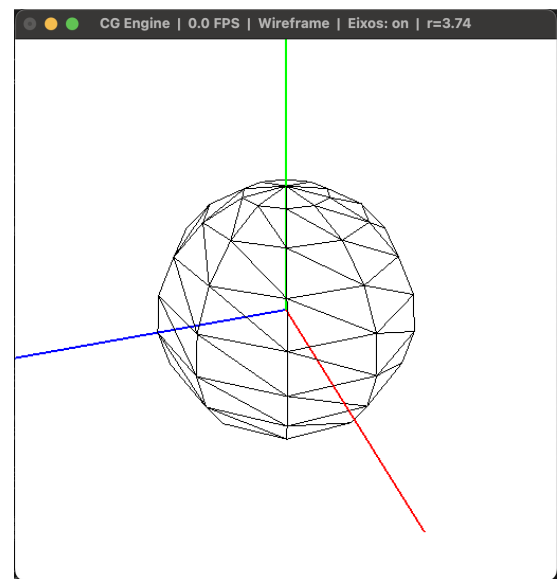


(b) Resultado Obtido

Figura 12: Teste 2 - Cone

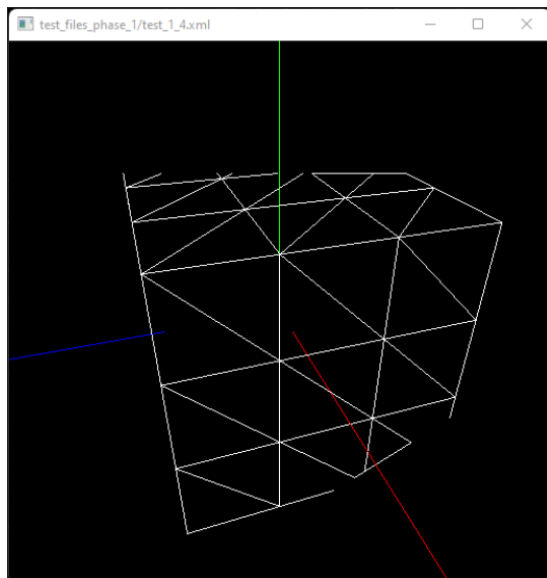


(a) Resultado Esperado

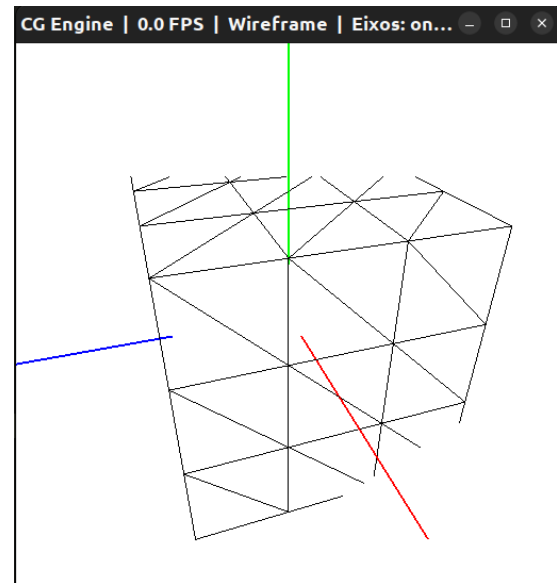


(b) Resultado Obtido

Figura 13: Teste 3 - Esfera

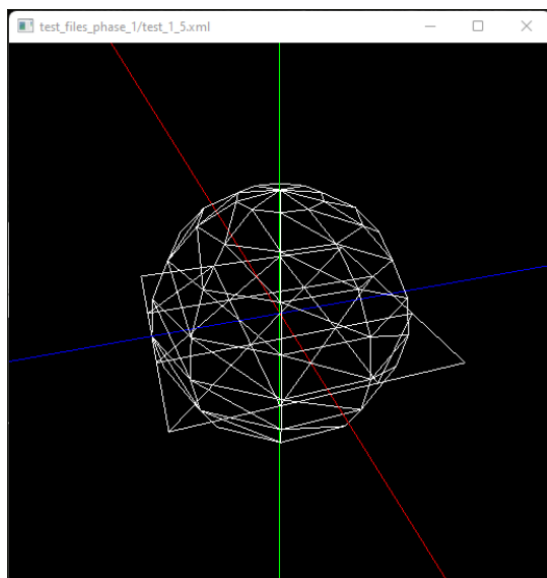


(a) Resultado Esperado

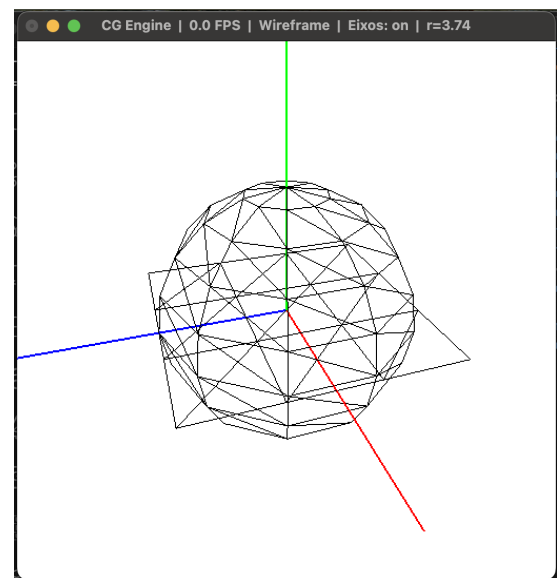


(b) Resultado Obtido

Figura 14: Teste 4 - Caixa



(a) Resultado Esperado



(b) Resultado Obtido

Figura 15

6 Conclusão

A Fase 1 cumpre os requisitos do enunciado: o *generator* produz ficheiros .3d para as quatro primitivas obrigatórias, plano, caixa, esfera e cone, tendo sido ainda implementadas duas primitivas adicionais, o *cilindro* e o *torus*. O *engine* lê a cena a partir de um ficheiro XML, carrega os modelos e renderiza-os com suporte a três modos de visualização e navegação interativa completa por teclado e rato.

Todas as primitivas produzem geometria com *winding order* CCW e normais calculadas analiticamente, armazenadas no formato .3d desde esta fase. O modelo de câmara em coordenadas esféricas (α, β, r) fornece desde já uma experiência de navegação completa, com órbita livre, zoom e movimento da mesma.